

- **Reference parameters**, i.e., parameters declared by the keyword **byref**, cannot have a default argument.

### See also

[Declare Parameters](#)

[Colors for Syntax Highlighting](#)

[Video on YouTube](#)

<https://youtu.be/5t-wLNmKpbU?si=4oreA6Pt8xKQtFuP&t=126>

## General Access to SimTalk

### General Access to SimTalk

The programming language *SimTalk* extends the ways you can model and control your simulation. Each object has built-in properties providing many useful features.

#### Remarks

If your model requires more detailed or completely different properties, you will program these in the programming language *SimTalk*. You can also combine the *Method* object with the built-in objects to create models of great complexity.

First, you will type the statements, which the built-in *Interpreter* executes, into an instance of the object **Method M**. Then press the **F7** and **F5** keys to execute your source code.

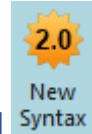
The **Copilot** can assist you in writing your source code in a *Method*.

When you then run the simulation, the *Interpreter* executes the source code, which you typed into your *Method* line-by-line and takes the actions you programmed.

#### Note

*SimTalk* normally does not distinguish between upper- and lower-casing for the names of *methods*, *attributes* and *read-only attributes* which you type into the source code of your *Method* objects.

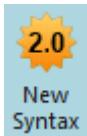
**SimTalk 2.0** makes programming methods in *Plant Simulation* faster, easier, and less error-prone than **SimTalk 1.0**. New simulation models use the **SimTalk 2.0 notation** by default!



You can change between **SimTalk 1.0** and **SimTalk 2.0** by clicking **New Syntax [command]** on the **Tools** ribbon tab of the *Method* you are programming. If you want to use **SimTalk 2.0** syntax for all new *Methods* which you are going to program, activate **New Syntax** in the *Method class* in the *Class Library*. Changes caused by **SimTalk 2.0** also affect how the window **Show Attributes and Methods** shows the signature of *methods*, *attributes* and *read-only attributes*.

**Note**

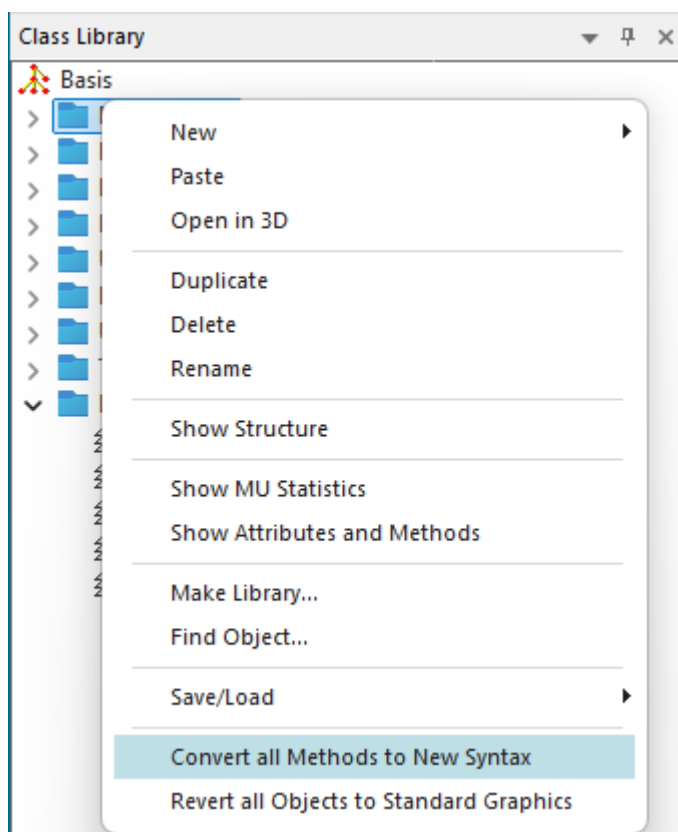
You can select if you want to use the **SimTalk 2.0 notation** or the **SimTalk 1.0 notation** for each and every one of your *Methods*. You can also freely mix **SimTalk 2.0 notation** and **SimTalk 1.0 notation** in your simulation models. There is no need to reprogram your existing source code.

**Note**

Clicking **New Syntax** in an existing *Method*, which you programmed in **SimTalk 1.0 notation**, automatically converts the source code to the correct **SimTalk 2.0 notation**.

**Note**

To convert the source code of all existing *Methods* in a simulation model to the new syntax, hold down **Shift**, click the object **Basis** in the *Class Library* with the right mouse button and click **Convert all Methods to New Syntax**.



## See also

[Why Use SimTalk 2.0?](#)

[SimTalk Access to 3D Functions](#)

[String \[SimTalk\] - value](#)

## Why Use SimTalk 2.0?

*SimTalk 2.0* provides a number of significant improvements compared to *SimTalk 1.0* making programming easier and more comfortable. Use *SimTalk 2.0* source code whenever possible and be prepared that support for *SimTalk 1.0* might be terminated at a future date.

### Remarks

*SimTalk 2.0* encompasses:

- **A line-controlled syntax**

You do not have to type a semicolon `;` at the end of each statement. Instead, you can type a single statement into a line. As you might want to be able to split a statement and distribute it over several lines, the interpreter automatically determines if the statement is still incomplete and, if this is the case, continues it in the next line. You can type a semicolon after a statement to add another statement to the same line though.

| SimTalk 2.0                                 | SimTalk 1.0                                  |
|---------------------------------------------|----------------------------------------------|
| <code>MyStation.setName("MyStation")</code> | <code>MyStation.setName("MyStation");</code> |

- **A simplified body syntax**

In *SimTalk 1.0* the source code requires the keywords **is do end**. *SimTalk 2.0* does not need these keywords.

In *SimTalk 2.0* you declare **parameters** with the keyword **param**, **local variables** with the keyword **var**, and the **return value** of a method with the keyword **->** (hyphen plus right angle bracket).

| SimTalk 2.0                                 | SimTalk 1.0                                                            |
|---------------------------------------------|------------------------------------------------------------------------|
| <code>MyStation.setName("MyStation")</code> | <code>is<br/>do<br/>    MyStation.setName("MyStation");<br/>end</code> |

- **Improved referencing of methods and global variables**

In *SimTalk 1.0* *Methods* and *Variables* are referenced with the reference operator **ref**.

In *SimTalk 2.0* you reference *Methods* and *Variables* with a leading **& operator**, for example `var o:object := &Method`.

- **New div/mod operator:** In *SimTalk 1.0* the **// operator** represents an integer division and the **\ operator** represents an integer modulo operation.

In *SimTalk 2.0* the keyword **div** represents an **integer division** and the keyword **mod** represents an **integer modulo operation**.

- **Improved string literals**

In *SimTalk* 2.0 a backslash (\) inside a **string** only protects double quotation marks and line breaks, it does not protect another backslash as it did in *SimTalk* 1.0.

"A *string literal* is a sequence of characters from the source character set enclosed in double quotation marks ("). To represent a double quotation mark in a string literal, use the escape sequence \". The backslash \ must be followed with a second backslash \\ when it appears within a string." Microsoft Learn

```
a := "It is \"very\" urgent." -- the backslash protects the quotation
marks
Path := "C:\Temp"           -- no need to protect the backslash here
Path := "C:\\\" + FolderName -- protected backslash at the end of the
string
```

- **Time literals**

A *time literal* starts with a digit and must contain one or more colons. A *time literal* can contain a decimal point and decimal places.

```
wait 1:30 // wait for 1 minute and 30 seconds
&Methode.methCall(1:0:0:0.5) // call the method in 1 day and half a
second
```

- **JSON Literals**

*JSON literals* are enclosed in curly braces {} and contain JSON elements separated by commas. The JSON elements consist of a string literal, a colon, and a JSON value.

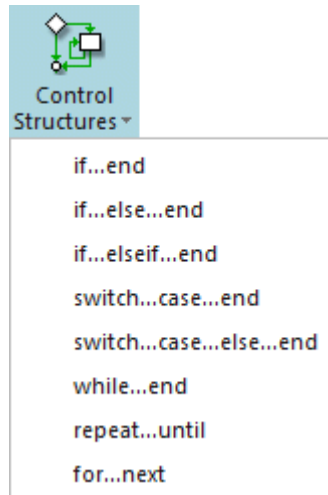
```
var j: json
var jobTitle: string := "Engineer"
var addressArray: string[] := ["Main street 42", "10001 New York"]
j := { "name": "Alice", "age": 42, "female": true,
      "job": jobTitle, "address": addressArray }
```

- **Default arguments**

Define default arguments for formal parameters by typing an assignment operator and the default value after the parameter declaration, for example: param x := 0.

- **Simplified control flow statements**

The control structures which you can insert with the command **Structure** into your source code, reflect the simplified control structures.



You can type control flow statements using the simplified syntax:

- **if-end** instead of **if-then-end**
- **for-next** instead of **for-loop-next**
- **switch-case-end** instead of **inspect-when-then-end**
- **while-end** instead of **while-loop-end**

The keyword **then** in **if-statements** is optionally allowed.

- The keyword **loop** in **loops** is optionally allowed.
- We added the keyword **continue**, which skips the rest of the loop iteration, and continues with the next iteration. If the iteration was the last iteration, **continue** exits the loop.
- A **changed about equal operator**

You now have to use:

- **~=** instead of **==**, compare **Tolerance for about equal (~=) comparison**.
- **<~=** instead of **<==**
- **>~=** instead of **>==**
- **New operators for adding, subtracting, or multiplying a value**
  - **x += y** is short for **x := x + y**
  - **x -= y** is short for **x := x - y**
  - **x \*= y** is short for **x := x \* y**
- **An improved list and table syntax**

You now only type list ranges using braces **{}**. The optional 1.0 syntax using a grave accent **`[]** is no longer available.

- Read an element of a **one-dimensional list** with the **bracket operator** `[]`. This works for *StackFile* and *QueueFile* as well. You can read and remove an element from a *DataList* with the built-in method [remove](#).

**Note**

When reading the contents of a cell of a *DataList* with the **bracket operator** `[]` in *SimTalk 1.0*, this reads and removes the contents of the designated cell. The remaining cells move up by one position.

When assigning a value to the designated cell with the **bracket operator** to a *DataList* in *SimTalk 1.0*, this value is inserted into the cell and the existing cells moves down by one position.

**Note**

When reading the contents of a cell with the **bracket operator** `[]` in *SimTalk 2.0*, the contents of the designated cell will be read, but will remain in the *DataStack*, *DataQueue*, and the *DataList*.

When assigning a value to the designated cell with the **bracket operator** to a *DataList* in *SimTalk 2.0*, this value will just overwrite the contents of the existing cell. This way a *DataList* will behave the same way as a *DataTable* with one column.

**See also**

[SimTalk Access to 3D Functions](#)

[String \[SimTalk\] - value](#)

## Introducing SimTalk

*SimTalk* consists of the built-in *methods*, *read-only attributes*, and *attributes* and of *control structures*.

### Remarks

*SimTalk* encompasses:

- The built-in *methods*, *read-only attributes*, and *attributes* which the built-in *Plant Simulation* objects provide. These are defined by our software engineers and offer a base you can build on.
- Control structures and language constructs. With loops and conditional branching you can control the sequence of method execution.

We use this notation when describing the **syntax** of a method:

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

#### Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

If a parameter has a default value, the signature shows the default value after the parameter

If the *method* has a return value, the signature shows its data type after the arrow ->.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



| Name         | Value | Inherited | Watchable | Signature                                                       |
|--------------|-------|-----------|-----------|-----------------------------------------------------------------|
| contentsList | []    |           |           | ((Contents:table]) -> any                                       |
| createAttr   |       |           |           | (NameOfUserDefinedAttribute:string, DataType:string) -> boolean |
| createIcon   |       |           |           | ((IconName:string, Width:integer, Height:integer]) -> integer   |
| deleteAttr   |       |           |           | (NameOfUserDefinedAttribute:string) -> boolean                  |

Name of  
the method

Identifier and data type of  
the parameter you enter

Data type of the  
return value

| Name         | Value | Inherited | Watchable | Signature                      |
|--------------|-------|-----------|-----------|--------------------------------|
| moveToFolder |       |           |           | (Folder:object) -> object      |
| MU           | VOID  |           |           | ((Index:integer:=1]) -> object |
| MUPart       | VOID  |           |           | ((Index:integer:=1]) -> object |

Name of  
the method

Identifier and data type of  
the parameter you enter

Default  
value

Data type of the  
return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

In the *Plant Simulation Help* we use these conventions:

- Within descriptions in the body text on the topic pages the names of objects begin with an upper-case letter and are italicized, for example *ParallelStation*, *Station*, etc.

In the *Plant Simulation Help*, the colored term sometimes is a link to an object, for example [ParallelStation](#). Click the link to open that topic in the *Online Help*.

- In the description in the *Plant Simulation Help* the names of *methods* begin with a lower-case letter. Each new term after that begins with an upper-case letter. *Methods* are italicized, for example *derive*, *updateDialog*, etc.

In the *Online Help*, the colored term sometimes is a link to a *method*, for example [derive](#). Click the link to open that topic in the *Online Help*.

- In the description in the *Online Help* the names of *attributes* and *read-only attributes* begin with an upper-case letter. Each new term after that begins with an upper-case letter as well. *Attributes* and *read-only attributes* are italicized, for example *CreationTableActive*, *ReferenceTime*, etc.

In the *Online Help*, the colored term sometimes is a link to an attribute, for example [CreationTableActive](#). Click the link to open that topic in the *Online Help*.

- In the description in the *Online Help* we use *MU* and *part* interchangeably, for example the processed *MU* and the processed *part* mean the same. If *Part* start with an upper-case P though, it refers to the *MU* of type **Part**.

### Note

These spelling conventions apply to the *Online Help*.

### Note

*SimTalk* normally does not distinguish between upper- and lower-casing for the names of *methods*, *attributes*, and *read-only attributes*, which you type into the source code of your **Method** objects. You do not have to use the spelling which the window **Show Attributes and Methods** shows. You can, for example, type in `CreationTableActive`, `creationtableactive`, or `CREATIONTABLEACTIVE`.

### Note

*SimTalk* supports **English** and **German** for programming the source code of your methods.

For this reason the **German Help** shows the **English** name of the *attribute*, *read-only attribute*, and *method* next to the **German** name, separated by a forward slash, for example **EnergieAktiv [SimTalk] / EnergyActive**. This facilitates programming methods if you are working with the **German** version of *Plant Simulation* and are creating an **English** language model that your colleagues in other countries can use as well.

## EnergieAktiv [SimTalk] / EnergyActive

Aktiviert die Energieeinstellungen für das mit <Pfad> bezeichnete Objekt (**true**) oder deaktiviert diese (**false**).

### Typ

Attribut

### Syntax

```
<Pfad>.EnergieAktiv:boolean
```

 Kopieren

Compare [Creating a German Model with English Identifiers](#).

## SimTalk Attributes, Read-only Attributes, Methods, and Functions

*SimTalk* provides *attributes*, *methods*, *read-only attributes*, and *functions*.

### Remarks

The description of the individual *attributes*, *methods*, *read-only attributes*, and *functions* shows this under the section title **Type** in the *Online Help*.

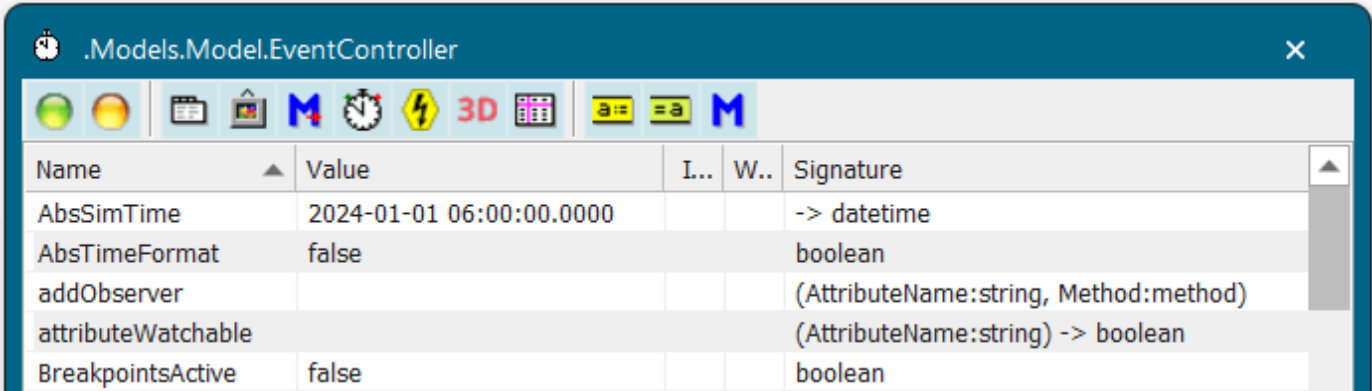
### AbsTimeFormat [SimTalk]

Shows the time of the *EventController* designated by <Path> as the absolute time (`true`) or as the relative time (`false`).

### Type

Attribute

The window **Show Attributes and Methods** shows most types in the column **Signature**.



| Name               | Value                    | I... | W.. | Signature                             |
|--------------------|--------------------------|------|-----|---------------------------------------|
| AbsSimTime         | 2024-01-01 06:00:00.0000 |      |     | -> datetime                           |
| AbsTimeFormat      | false                    |      |     | boolean                               |
| addObserver        |                          |      |     | (AttributeName:string, Method:method) |
| attributeWatchable |                          |      |     | (AttributeName:string) -> boolean     |
| BreakpointsActive  | false                    |      |     | boolean                               |

### Attributes in SimTalk

An *attribute* in *SimTalk* sets or gets a property of an object.

*Attributes* encompass the built-in attributes of the object, user-defined attributes, local variables, and objects of type *Variable*.

An *attribute* in *SimTalk* looks like this:

```
EventController.AbsTimeFormat := true
```

```
print EventController.AbsTimeFormat // returns true
```

## Methods in SimTalk

A *method* in *SimTalk* is a block of source code that executes a task when called.

*Methods* as a rule have parameters, have side effects, and can return a value.

A *method* in *SimTalk*

- gets information from an object and returns a value
- computes a value
- starts one or several actions that control the behavior of the object

A *method* in *SimTalk* looks like this:

```
TableVariable := EventController.getEventList(-1)
```

```
MyStation.addObserver("occupied", &myMethod)
```

## Read-only Attributes in SimTalk

A *read-only attribute* in *SimTalk* returns a value of a property of an object.

*Read-only attributes* do not expect parameters, do not have side effects, and return a value.

A *read-only attribute* in *SimTalk* looks like this:

```
param sensorID: integer, Front: boolean
if @.ID = 1
  @.Speed := 0
  print EventController.SimTime, " The first Transporter stopped."
end
```

## Functions in SimTalk

A *function* in *SimTalk* is a piece of source code that applies in general and is not object-specific.

A *function* in *SimTalk* looks like this:

```
if currentEventCtl /= VOID
  print "simulation running in frame", currentEventCtl.location
  print "simulation running in frame", root
end
```